

CAREFUL: a Secure and Privacy-preserving Deletion Notification Distribution Protocol

Qipeng Song, Xin Deng, Yue Li*, Zhihao Dong, Xingyue Zhu, Hui Li

School of Cyber Engineering, Xidian University, Xi'an 710126, China

{qpsong, liyue}@xidian.edu.cn, {xindeng, dongzhihao, xingyuezhu}@stu.xidian.edu.cn, lihui@mail.xidian.edu.cn

Abstract—The right to deletion mandates that data controllers receiving deletion requests not only erase the specified data but also notify other controllers to do the same. Designing a secure and privacy-preserving protocol for distributing deletion notifications across involved data controllers, a topic not previously addressed in the literature, presents significant challenges. In this paper, we introduce CAREFUL, a secure and privacy-preserving deletion notification distribution protocol—the first to address these challenges. It operates within a centralized architecture consisting of regulatory and service planes. The fundamental principle of CAREFUL is that the regulatory plane creates a cryptographic access control structure, ensuring that data controllers in the service plane only identify the next-hop nodes for notification. With CAREFUL, the circulation history of data pending for deletion, which is a special user's privacy, can be preserved. Moreover, CAREFUL protects the deletion notification distribution from various malicious attacks over untrusted underlay networks. Experimental results validate CAREFUL's practicality and its efficiency in terms of resource overhead.

Index Terms—Right to Deletion, Deletion Notification Verification, Privacy Protection

I. INTRODUCTION

In today's digital society, data is extensively and universally shared among data controllers, including public institutions and private enterprises, for a variety of purposes. However, this pervasive dissemination of data poses significant challenges to the protection of an individual's right to deletion, a right acknowledged by numerous privacy protection laws and regulations such as European Union's General Data Protection Regulation (GDPR). The enforcement of the right to deletion mandates that the data controller, upon receiving a request for data deletion from an individual, must not only erase the requested data but also undertake reasonable measures to notify other data controllers to delete the same data [1].

The absence of a robust technical system to safeguard the right to deletion significantly impedes data sharing among data controllers, as non-compliance with this right poses substantial penalty for those failing to erase the required data as mandated. Despite attracting considerable research attention, the related studies predominantly concentrate on verifiable data deletion within a singular data controller's domain. In essence, these studies aim to ensure the permanent and irreversible elimination of the targeted data. In digital society characterized with data circulation, when enforcing the right to deletion, a critical preliminary step before actual data deletion is the distribution of deletion notifications, i.e.,

notifying all involved data controllers regarding the specific user and data to be erased. However, this aspect has never been the focus of research efforts before this paper, despite its equal importance to the issue of verifiable data deletion.

The design of a deletion notification distribution system is confronted with several challenges: 1) Compliance and security of the deletion notification distribution process: Safeguarding against malicious attacks as deletion notifications navigate through untrusted networks, so that all relevant data controllers, who possess the data slated for erasure, are accurately informed; 2) Minimization of resource overhead: Given that adherence to the right to deletion does not inherently yield economic benefits, a mechanism that is both secure and lightweight is more likely to be adopted by data controllers; 3) Prevention of privacy leakage: It is imperative to prevent privacy breaches that could arise from deletion notifications, which might reveal users' data circulation history and, by extension, their preferences. It is crucial to ensure that each data controller is only made aware of its immediate predecessor and successor during notification process.

As an initial response to the challenges outlined, this paper introduces CAREFUL: a seCure And pRivacy-preserving dEletion notiFication distribUtion protoCoL. CAREFUL is tailored for a centralized regulatory framework, comprising regulatory and service planes. It is a cross-plane collaborative approach that requires the intervention of regulatory and service planes. The core idea is that the regulatory plane utilizes its global view about the circulation history data to be deleted. It constructs a cryptographic access control structure that ensures data controllers service plane only and can only determine the identities of the next nodes for notification, provided the deletion notification has been accurately forwarded by the legitimate prior node.

In summary, the main contributions of this paper are as follows:

- We propose CAREFUL, a novel protocol tailored for a centralized privacy protection regulatory framework. It ensures the distribution of deletion notifications among involved data controllers, in a secure, privacy-preserving yet lightweight manner.
- We propose a hierarchical feedback mechanism for acknowledging deletion notification message and a centralization-based anomaly localization to locate the suspect data controllers.
- We conduct a comprehensive security analysis and eval-

uation upon a CAREFUL to evaluate the compliance cost brought to the central node and data controllers.

The rest of this paper is organized as follows: Sec. II survey the related work. Sec. III presents the target scenario and threat model. Sec. IV elaborates our proposal. Sec. V provides a security analysis and the performance evaluations for CAREFUL. Sec. VII concludes this paper.

II. RELATED WORK

In recent years, as citizens become increasingly aware of their right to deletion, research about right to deletion protection has attracted lots of attention. The existing studies about right to deletion protection can be broadly classified into the following categories.

Irrecoverable data deletion. Researchers have explored various methods to permanently erase data from physical media. Reardon et al. [2] investigate the approaches for irrecoverable data deletion at different interfaces to physical medium, and present a taxonomy of adversaries with different capabilities. Xiong et al. [3] present a key derivation encryption algorithm and construct a secure data deletion scheme for encrypted data deletion. Gnatyuk et al. [4] propose a randomized overwriting based method for guaranteed data deletion. Lee et al. [5] propose a secure yet lightweight deletion method for USB flash storage. Li et al. [6] design a secure user space data deletion method for Android. Z Ratliff et al. [7] present a new software-level approach named HOLEPUNCH, designed to provide secure file deletion with crash consistency.

Verifiable data deletion. This research direction aims to validate for the data owner that the required data has been conclusively erased. Dong et al. [8] introduce a trusted deletion system model for cloud storage, incorporating techniques for verifiable deletion and proposing various methods for timely and fine-grained deletion verification. Yang et al. [9] offer a blockchain-based scheme for the publicly verifiable deletion of personal information. Hao et al. [10] leverage TPM and cryptographic technologies, and design a mechanism that enables public verification of deletion operations. Ma et al. [11] devise an attribute encryption verifiable data deletion scheme. To better suit IoT devices, Hall et al. [12] present a hybrid deletion verification technique on private cloud hosts, combining password protection and overwriting.

Machine unlearning. Machine unlearning, which aims to eliminate the influence of deleted data on trained AI models. Bourtole et al. [13] propose the SISA scheme for a fast model retraining without deleted data. Eisenhofer et al. [14] conceptualize a novel cryptographically verifiable definition of forgetting and demonstrate its practicality.

In summary, the related studies primarily focus on methods for irrecoverably and verifiably erasing data from physical media. To our best knowledge, our paper is the first effort to tackle the issue of cross-domain deletion notification distribution prior to actual data deletion.

III. PROBLEM STATEMENT

A. Target Scenario

We consider the following scenario: a central node is assumed to be deployed, responsible for documenting the data circulation records for all regulated data controllers. The rationale behind this is that, in data exchange infrastructures such as data spaces or data marketplaces, one of the privacy-preserving requirements is to ensure the transparency and traceability of data exchange and data use. Therefore, data controllers are required to report logs of data transactions to regulatory agencies or data brokers, enabling the generation and management of data circulation records. The design of a data circulation recording mechanism itself has become a significant research topic in recent years [15]–[17] and will be the focus of our future work.

When enforcing the right to deletion, a user initiates a deletion request to the data controller (i.e., source data controller) possessing the data for removal. The latter then requests the deletion notification tree from the central node, a spanning-tree like data structure that represents the circulation history of the data for deletion. Upon receiving the deletion notification tree, the data controller determines the downstream data controllers in the deletion notification tree, generates and dispatches the appropriate deletion notifications. This process continues iteratively until all leaf nodes¹ within the deletion notification tree have been duly notified.

B. Threat Model

The deletion notification system of data controllers can be compromised, leading to the following threats:

Tampering Attack: The deletion notification message is maliciously tampered with, resulting in either the unintended deletion of data or the erroneous notification of an unintended data controller for data deletion. Additionally, it could lead to the omission of notifications to involved data controllers.

Denial of Service (DoS) Attack: Attackers abuse the right to deletion by flooding the system in charge of processing deletion notifications with an excessive number of deletion requests. This flood overwhelms the victim data controllers, rendering them unable to handle the legitimate deletion requests from regular clients.

Path Privacy Breach: An attacker intercepts the deletion notification and analyzes its contents, aiming to trace the circulation path of a specific user's private information. This data circulation path can reveal the user's preferences, constituting a form of privacy that belongs to the user.

Notification Drop Attack: Deletion notifications may be intentionally discarded by an attacker before reaching the next hop, causing downstream data controllers to violate an individual's right to deletion.

IV. DESIGN

This section first provides an overview of our proposal, followed by a detailed presentation of the key steps involved.

¹ data controller and node are interchangeably used in this paper.

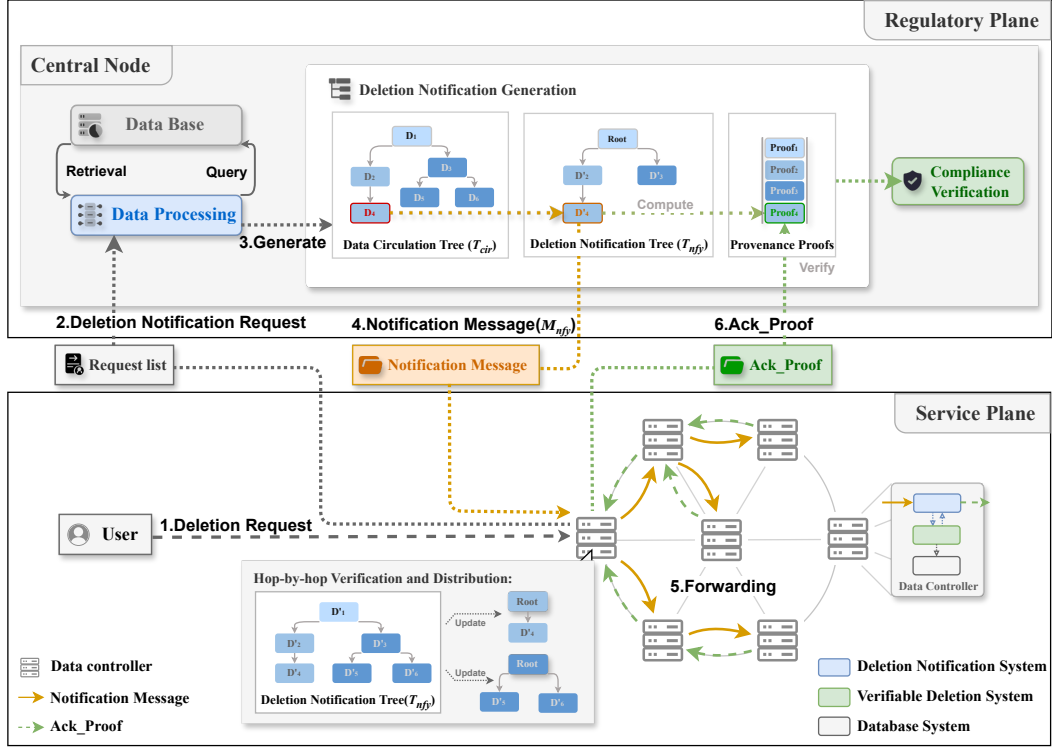


Fig. 1: Overview of workflow for CAREFUL

As illustrated in Fig. 1, the workflow of our design proceeds as follows: Upon receiving a deletion request, the source data controller initiates a message containing the payload of the user's deletion request $\mathcal{M}_{req}$ (for example, a globally unique ID for the data to be deleted), which is sent to the central node. The central node then queries its internal database for data circulation records and reconstructs a data circulation tree $\mathcal{T}_{cir}$. Based on $\mathcal{T}_{cir}$, the central node generates a deletion notification message $\mathcal{M}_{nfy}$ for the source data controller.

Each data controller involved in the process, upon receiving the deletion notification from its predecessor, decrypts the encrypted IDs of downstream data controllers to be notified following successful provenance authentication and forwards the updated deletion notification message. This approach ensures compliance with the distribution path and protects path privacy during the deletion notification distribution phase at the service plane. It is important to note that the permanent erasure of data is not within the scope of this paper.

When a leaf node is duly notified, it initiates a backward acknowledgement process towards the source data controller. Ultimately, the central node can verify whether all involved nodes have been correctly notified in a secure, lightweight, and privacy-preserving manner.

Our solution can be implemented within a trusted execution environment (e.g., Intel's SGX) to further enhance its robustness.

A. Deletion Notification Generation

In this step, leveraging its global view of the data circulation history, the central node generates a cryptographic access control structure named *the deletion notification tree* $\mathcal{T}_{nfy}$. This structure allows each node to verify the authenticity of its provenance and then decrypt the IDs of downstream nodes. Based on the initial deletion request message $\mathcal{M}_{req}$, the deletion notification tree, and additional necessary information, the central node crafts the deletion notification message $\mathcal{M}_{nfy}$. The entire process is outlined in Alg. 1.

Lines 3 – 6 in Alg. 1 is the initialization stage. To facilitate the tree traversal, *queue* holds pairs constituted by D_i and its parent D_i^p . For root D_1 (i.e., source data controller), its parent node is the central node. To support the provenance authentication, we introduce a hash-table like data structure *provenance_proofs* for saving the provenance proof for each node. This data structure is also useful for the final notification acknowledgement verification. The provenance proof of D_1 is a random value, since it directly receives deletion notification message from the trusted agency. For $\mathcal{T}_{nfy}$, each node except the root saves D_i^c , the encrypted ID of D_i . The root save ID of the central node and corresponding provenance proof required by decryption of D_i^c .

The construction of $\mathcal{T}_{nfy}$ entails the calculation of D_i^c by traversing $\mathcal{T}_{cir}$, outlined in lines 7 – 24. For each node D_i , a session key sk_i is derived by $sk_i = \mathcal{H}(K_i || T_s)$, where $\mathcal{H}$ is cryptographic hash function, T_s is the time stamp for defending reply attack, K_i is the master key pre-shared between the

D_i and the central node. A secure channel is assumed to be available for negotiating K_i when regulated data controllers register to the central node. To ensure simultaneously integrity of $\mathcal{M}_{req}$ and provenance proof, *valid_proof* is obtained with $valid_proof_i = \text{HMAC}(sk_i, \mathcal{M}_{req} || provenance_proof_i)$.

If D_i is not leaf node, we iterate its child set calculate the key for encrypting the ID of child D_j as outlined by line 14-15. The provenance of D_j is calculated by $\text{HMAC}(sk_i, D_j^c)$. In case D_i is leaf node, so we let it encrypt its own ID.

Algorithm 1 Deletion Notification Generation

```

1: Input:  $\mathcal{T}_{cir}, M_{req}, T_s$ 
2: Output:  $M_{nfy}$ 
3:  $\mathcal{T}_{nfy}.root.id \leftarrow$  ID of the central node
4:  $\mathcal{T}_{nfy}.root.value \leftarrow$  random value
5:  $queue \leftarrow [(D_1, \mathcal{T}_{nfy}.root.id)] \triangleright$  for traversal of  $\mathcal{T}_{cir}$ 
6:  $provenance\_proofs \leftarrow \{D_1 : random\} \triangleright$  initialize dict
7: while queue not empty do
8:    $(D_i, D_i^p) \leftarrow queue.pop()$ 
9:    $sk_i \leftarrow \mathcal{H}(K_i || T_s)$ 
10:   $provenance\_proof_i \leftarrow provenance\_proofs[D_i]$ 
11:   $valid\_proof_i \leftarrow \text{HMAC}(sk_i, M_{req} || provenance\_proof_i)$ 
12:  for  $j, D_j$  in child set of  $D_i$  do
13:    if  $D_j$  is not none then
14:       $queue \leftarrow (D_j, D_i)$  on  $\mathcal{T}_{cir}$ 
15:       $key_j \leftarrow \text{HMAC}(sk_i, D_i^p || valid\_proof || T_s || j)$ 
16:       $D_j^c \leftarrow \text{Enc}(key_j, D_j)$ 
17:       $provenance\_proofs[D_j] \leftarrow \text{HMAC}(sk_i, D_j^c)$ 
18:    else
19:       $key_j \leftarrow \text{HMAC}(sk_i, D_i^p || valid\_proof || T_s)$ 
20:       $D_j^c \leftarrow \text{Enc}(key_j, D_i)$ 
21:    end if
22:     $\mathcal{T}_{nfy}.insert(D_i, D_j^c) \triangleright$  add encrypted ID of  $D_j$  as leaf node of  $D_i$  on  $\mathcal{T}_{nfy}$ 
23:  end for
24: end while
25: add  $T_s, \mathcal{T}_{nfy}, M_{req}$  into  $M_{nfy}$ 
26: return  $M_{nfy}$ 

```

B. Hop-by-hop Verification and Distribution

The hop-by-hop forward verification and distribution of deletion notification is described by Alg. 2.

For a data controller D_i , upon reception of deletion notification message from its precedence, it first takes out time stamp T_s and verify its validity. Then D_i retrieves $\mathcal{T}_{nfy}$ from $\mathcal{M}_{nfy}$. The ID of parent node (i.e., the ID of precedence data controller) and provenance proof, can be retrieved from root node of $\mathcal{T}_{nfy}$. Consequently, the session key sk_i is derived to calculate *valid_proof*.

D_i iterates the child node of root on $\mathcal{T}_{nfy}$ and derives the decryption key for each child D_j^c as follows:

$$key_j = \text{HMAC}(sk_i, D_i^p || valid_proof || T_s || j). \quad (1)$$

Note that if valid proof is maliciously tampered or forged, decryption key cannot be obtained with success, which leads to the failure of parsing the ID of next hop. This step simultaneously achieves provenance authentication and ID decryption. In addition, D_i cannot get the ID of nodes other than its childs.

If decrypted ID D_j is different from that of current data controller D_i , this indicates that current data controller is the not the leaf on $\mathcal{T}_{nfy}$. Hence, D_i starts the process for generating and forwarding the deletion notification message for child node D_j (shown in line 16-18). Otherwise, the current data controller initiates the backward acknowledgement process detailed in Sec. IV-C.

Algorithm 2 Deletion Notification Verification and Distribution

```

1: Input:  $M_{nfy}$  from predecessor node
2: Output:  $M_{nfy}$  to successor node
3:  $T_s \leftarrow$  get time stamp from  $M_{nfy}$ 
4: if  $T_s < \text{current\_time} - \text{lifetime}$  then
5:   Drop the message
6: end if
7:  $sk_i \leftarrow \mathcal{H}(K_i || T_s)$ 
8:  $\mathcal{T}_{nfy} \leftarrow$  get deletion notification tree from  $M_{nfy}$ 
9:  $D_i^p \leftarrow$  root.id of  $\mathcal{T}_{nfy}$ 
10:  $provenance\_proof \leftarrow$  root.value of  $\mathcal{T}_{nfy}$ 
11:  $valid\_proof \leftarrow \text{HMAC}(sk_i, M_{req} || provenance\_proof)$ 
12: for  $j, D_j^c$  in child set of  $D_i$  do
13:   $key_j \leftarrow \text{HMAC}(sk_i, D_i^p || valid\_proof || T_s || j)$ 
14:   $D_j \leftarrow \text{DeEnc}(key_j, D_j^c)$ 
15:  if  $D_j \neq D_i$  then
16:    generate sub-tree  $\mathcal{T}_{nfy}(D_j)$  from  $\mathcal{T}_{nfy}$ 
17:     $\mathcal{T}_{nfy}(D_j).root.id \leftarrow D_i$ 
18:     $\mathcal{T}_{nfy}(D_j).root.value \leftarrow \text{HMAC}(sk_i, D_j^c)$ 
19:  else
20:    backward recursive acknowledgement process
21:  end if
22: end for

```

C. Backward Acknowledgement and Error Localization

To ensure the integrity and traceability of the deletion notification process, we design a hierarchical feedback mechanism for acknowledging deletion notification message. For leaf nodes, they immediately return back the required acknowledgement proof with the (2), to prevent from the tampering attack.

$$ack_proof = \text{HMAC}(sk_i, provenance_proof). \quad (2)$$

For nodes other than leaf nodes, after forwarding a deletion notification, they activate a timer to wait for acknowledgments from downstream nodes. Upon receiving the expected number of acknowledgments within the time limit, or if the timer expires, they generate their own acknowledgment proof, incorporating the proofs received, and report this to their upstream node.

Finally, the source data controller reports all the collected acknowledgement proofs to the central node. The latter is in charge of verifying the compliance of the whole deletion notification process.

During the deletion notification distribution process, nodes may encounter specific issues such as: 1) decryption failures for the IDs of the nodes to be notified; 2) failure to receive expected number of acknowledgments from downstream nodes before timer expiration. Both issues can result in an incomplete acknowledgment report to the central node, leading to verification failures. In such cases, the central node initiates root cause analysis for the involved data controllers. Detailed fault localization, however, is beyond the scope of this paper. Given that compromises in the deletion notification system are infrequent, this process would not overwhelm the central node.

V. EVALUATION

A. Security Analysis

This section analyzes why our solution is immune from the attacks identified in Sec. III-B.

Tampering Attack: With our design, if an attacker tampers a deletion notification (either message payload or deletion notification tree), the IDs of downstream nodes cannot be determined, thereby preventing them from forwarding the tampered notifications, and finally leading to the failure of compliance verification.

Denial of Service (DoS) Attack: Our design is a lightweight solution. For deletion notifications with an access control mechanism, attackers are unable to decrypt the deletion notification path, obtain the next forwarding path, acquire the next-hop sending address, or send an excessive amount of deletion notifications.

Path Privacy Breach: Our scheme ensures that each data controller, only and can only decrypt the IDs of next hops. Thus, attackers are not able to parse the whole data circulation history for data to be deleted.

Notification Drop Attack: Our solution incorporates a notification verification mechanism, ensuring the integrity of deletion notifications. Should an attacker attempt to drop a packet, the integrity check will fail, triggering an exception handling mechanism, such as resending, to mitigate the impact caused by the packet drop.

B. Implementation and Experimental Setting

We implement the functionalities of the regulatory and service planes of CAREFUL using the Python-based Flask framework and utilize a MySQL database to store data circulation records. Communication between the regulatory and service planes, as well as within the service plane, is facilitated by REST APIs.

Our experimental platform consists of two servers. The first one is a desktop with 32GB of memory, accommodating the functionality of the central node. The second server is a Dell T7920, which simulates a group of data controllers, and is equipped with an Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz and 256GB of memory. Both servers run Ubuntu

20.04 with the Linux 5.15.0-91-generic x86_64 GNU kernel. To simulate data controllers that exchange and share data, we rely on Docker containers. Each container runs a web server capable of receiving, processing, and forwarding deletion notifications. Since no data circulation recording mechanism is available in the real world, we simulate data exchange transactions on our platform, thereby generating the corresponding data circulation history, which is stored persistently in the MySQL database.

As this work is the first to address cross-domain deletion notification distribution, no state-of-the-art (SOTA) methods exist in the literature for direct comparison. Consequently, we select a TLS-based solution as the baseline for comparison. This choice reflects a common intuitive approach where each pair of adjacent nodes forwards encrypted deletion notifications using Transport Layer Security (TLS).

While TLS does not inherently address the issue of path privacy, it serves as a suitable benchmark for evaluating system resource consumption due to the absence of comparable protocols. By comparing against this baseline, we can effectively assess the efficiency and overhead of our proposed solution relative to a straightforward, widely understood method. Additionally, we evaluate the performance of both the central node and data controller processing programs, and further analyze the system resource consumption of the data controller program enhanced with Intel's Software Guard Extensions (SGX).

The experiments are designed to evaluate:

- 1) the performance and responsiveness of the regulatory plane, specifically including the time overhead of deletion notification generation, reflecting the burden imposed on the regulatory plane; the cumulative distribution function (CDF) of generation time for bunch of deletion notifications, which assesses responsiveness of central node; processing capability of central node, i.e., the number of deletion notification requests processed per second, reflecting serving capability of deletion requests from data controllers;
- 2) the performance and system resource consumption of the service plane, specifically including: processing capability, i.e., the number of deletion notifications processed per second, reflecting the performance and processing capacity of data controllers; and CPU and memory usage, which is used to evaluate the compliance cost borne by data controllers.

C. Performance Evaluation

Time overhead of deletion notification generation at regulatory plane. To evaluate the overhead on regulatory plane, we consider three kinds of tests: 1) "linear-tree" test, in which data is circulated along a chain formed by involved data controllers; 2) "rooted-tree" test, in which all nodes are children of root except itself; 3) "binary-tree" test, in which the data circulation tree is a binary tree. The deletion notification message generation algorithm (refer to Alg. 1) takes data circulation trees with different sizes as input. We

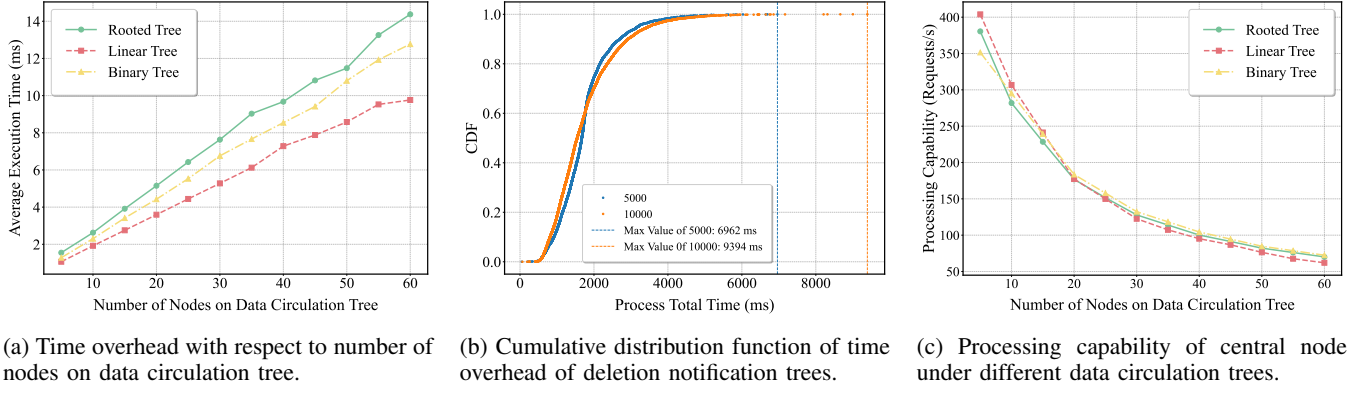


Fig. 2: Performance evaluation of central node.

repeat the experiment 50 times and measure the averaged execution time.

The evaluation result is illustrated in Fig. 2a. Across all tests, the time overhead for generating deletion notifications generally increases with the number of nodes in the deletion circulation tree. This increase is attributed to the computational complexity of the deletion notification message generation algorithm (see Alg. 1), which is $O(N)$, where N is the number of data controllers on the deletion circulation tree. The highest time overhead is observed in the rooted-tree test, primarily because, as per Alg. 1, each leaf node must generate an encrypted ID and a rooted data circulation tree has the most leaf nodes. This also explains why the linear-tree test exhibits the lowest time overhead.

The CDF analyze of the regulatory plane. To evaluate the response time of the regulatory plane, we perform a cumulative distribution function (CDF) test on its algorithm by sending the 5,000 and 10,000 deletion notification requests. As shown in Fig. 2b, the central node processes approximately 90% of the requests within 3,000 milliseconds. Despite the increase in request volume, the processing time only marginally increases, which demonstrates the system's scalability under varying loads. Since deletion notifications are less time-sensitive compared to tasks like video streaming, completing most requests within 3,000 milliseconds is considered acceptable.

Processing capability of regulatory and service planes. As shown in Fig. 2c, the processing capability of the regulatory plane gradually decreases as the number of nodes in the data circulation tree increases. This phenomenon is mainly due to the increased number of encryption operations as the number of nodes grows (the time complexity of the algorithm has been analyzed earlier). This enlarges the processing time and reduces the number of deletion notification requests processed per second. It is worth noting that the impact of different data circulation tree structures on processing capability is minimal because the regulatory plane only needs to execute encryption operations according to the inherent structure of the tree.

As shown in Fig. 3, the processing capability of the service plane diminishes as the number of nodes in the

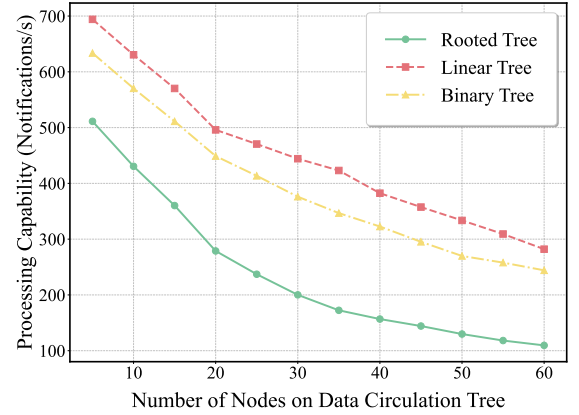


Fig. 3: Processing capability of service plane

data circulation tree increases. Unlike the regulatory plane, however, the service plane's processing capability is more heavily influenced by the number of child nodes at each node. The greater the number of child nodes, the lower the processing capability under similar conditions. The reason lies in that the number of forwarding operations is directly proportional to the number of child nodes. Additionally, the increase of nodes on data circulation tree leads to larger deletion notification packets, further impacting processing capability. However, in comparison to the regulatory plane, the impact of packet size on the service plane is less significant than the frequency of encryption.

System resource consumption at service plane. We conduct experiments under different network topologies to evaluate the service plane system resource overhead (CPU usage and memory footprint). We consider three representative network topologies: Karen, Lambdanet, and Viatel from Topology Zoo [18]. For each network topology, plenty of containers are launched to simulate the data controllers receiving and forwarding deletion notifications.

The experimental results are depicted in Fig. 4. CPU usage across various network topologies increases with the number of deletion notifications, as expected due to the greater resource demand on data controllers processing more notifications. The TLS-based solution shows the highest CPU

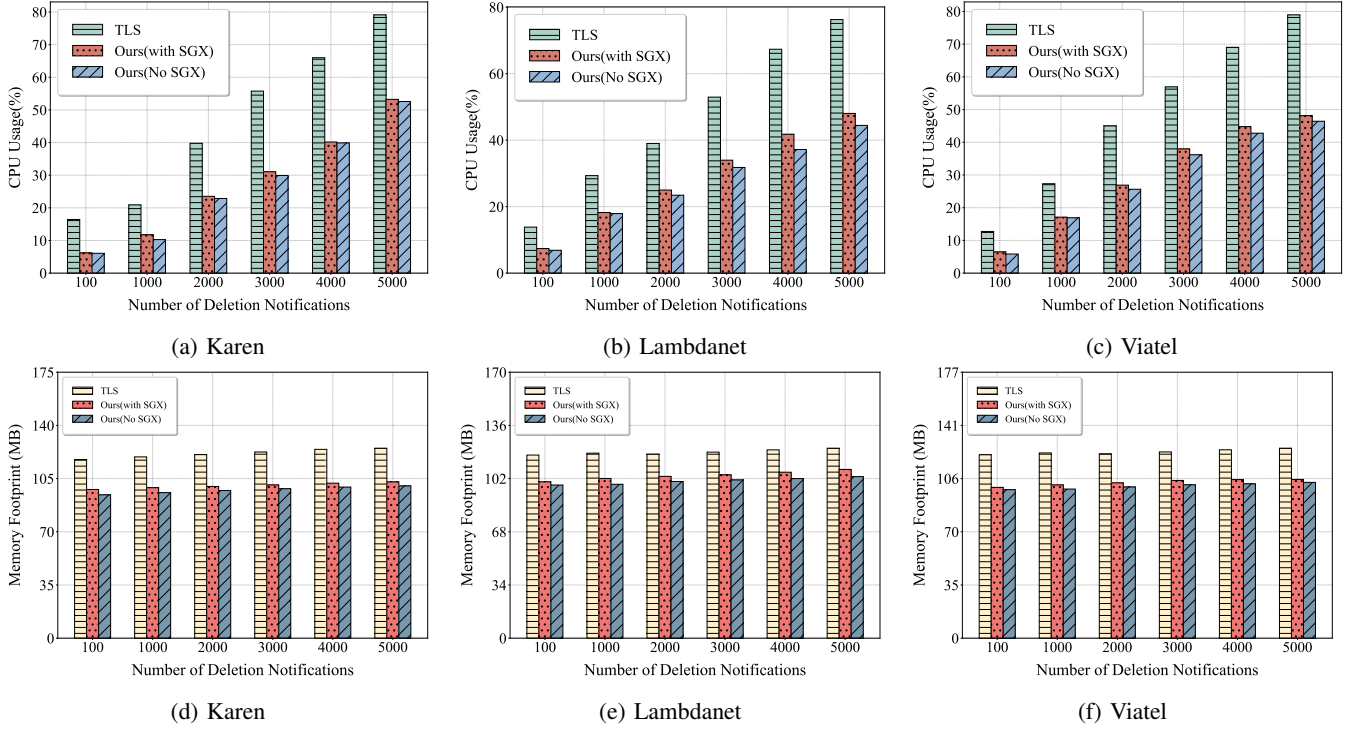


Fig. 4: Resource Utilization

consumption, primarily because it relies on computationally intensive asymmetric encryption and decryption operations, whereas our approach uses symmetric encryption, which significantly reduces CPU usage. Concerning the impact of SGX on CPU usage, implementing SGX results in a slight increase, mainly due to I/O operations between the enclave and non-enclave areas. This modest rise is because only essential functionalities are implemented within the SGX enclave, minimizing the computational load inside.

In terms of memory footprint, it can be observed that the TLS-based solution still ranks higher than other schemes, also due to its asymmetric encryption and decryption calculations. Using SGX does not significantly change memory consumption, as SGX is designed to enhance security at hardware level without consuming more memory resource. Unlike CPU usage, changes in the number of deletion notifications do not greatly affect memory footprint, which we attribute to the I/O-intensive nature of operations, where memory consumption is not heavily dependent on the volume of data. As data volumes increase, the significant rise in CPU usage is related to the use of asynchronous or non-blocking I/O, which enhances efficiency by allowing the CPU to handle other tasks while waiting for I/O completion. Therefore, as I/O operations increase, CPU usage also increases, thereby enhancing the overall performance of the application.

The comparison of results under different network topologies reveal that, the system resource overhead of our solution is independent of network topology. The reason behind lies in that our proposal employs a hop-by-hop verification and forward paradigm, making the performance overhead

independent of the network topology structure.

VI. DISCUSSION

A. Scalability Issues

One may challenge that a centralized architecture employed in this work is faced with scalability issues. This is a well-recognized challenge for all centralized systems. However, we argue that our approach can effectively address these concerns for several reasons:

- 1) **Algorithmic efficiency:** The algorithm deployed at the central node operates with a complexity of $O(N)$, where N represents the number of nodes in the data circulation tree. This linear complexity ensures that the computational load remains manageable even as the system scales. The linear growth of resource requirements means that our solution can handle increased demands without imposing excessive strain on the central regulatory authority;
- 2) **Unique Characteristics of deletion notification distribution:** different from high-bandwidth and long-lasting applications such as video streaming, the cross-domain deletion notification scenarios we target involve relatively low load demands. Our system handles deletion notifications, which are inherently lightweight and short-lived. Each notification is processed quickly, minimizing the burden on the central node and reducing the risk of performance bottlenecks;
- 3) **Best practices from established centralized systems:** We can leverage the advanced experiences of existing successful centralized architectures, such as load balancing, hierarchical architecture from DNS system, SDN

and LISP-empowered networks [19], to help alleviate scalability issues. By adopting these best practices, our solution can effectively mitigate potential single-point failures and scalability challenges inherent in centralized systems.

In summary, while scalability is a common concern for centralized architectures, the specific characteristics of our application, combined with algorithmic efficiency and the relevant best practices, provide a robust framework to address these issues effectively.

B. Applicability in Real World

Our solution is applicable for addressing the issue of cross-domain deletion notification distribution within data exchange infrastructures, such as the International Data Space (IDS) [20], which has gained significant attention in recent years.

In data-oriented infrastructures like IDS, trusted data controllers (data providers and data consumers in IDS terminology) are allowed to join the network, enabling the free exchange of data among different entities. The operation of the data space is required to be compliant with privacy protection regulations.

However, the technical procedure to protect an individual's right-to-deletion within IDS is still not fully established, which hinders the exchange of data. Within these data-centric infrastructures, a regulatory agency or a data broker designated by such an agency can take the role of the central node in our solution, as IDS requires data traceability and accountability during transactions. The regulatory agency or designated data broker has both the incentive and the capability to record data circulation history. It is worth noting that efficiently recording data circulation history remains a research challenge, which we will focus on in our future work.

When an individual exercises the right to deletion, the involved data controllers that deploy our solution can ensure the trustworthiness and completeness of cross-domain data deletion processes. Thus, data spaces can enhance their capabilities in managing data privacy and sovereignty, providing a reliable mechanism for both data providers and consumers to ensure compliance with data protection laws.

VII. CONCLUSION

CAREFUL represents the first initiative to ensure the secure and privacy-preserving distribution of deletion notifications. Our comprehensive security analysis confirms that CAREFUL effectively safeguards deletion notifications from tampering and maintains privacy within the data circulation tree. Performance evaluations demonstrate that CAREFUL is capable of handling a large volume of deletion notifications prompted by individual requests efficiently. For future work, we aim to address the efficient data circulation recording, which is an important prerequisite of our solution.

REFERENCES

- [1] Data Protection Commission, "The right to erasure (Articles 17 & 19 of the GDPR)," <https://www.dataprotection.ie/en/individuals/know-your-rights/right-erasure-articles-17-19-gdpr>, 2022, accessed: 01-30-2024.
- [2] J. Reardon, D. Basin, and S. Capkun, "Sok: Secure data deletion," in *2013 IEEE Symposium on Security and Privacy*, 2013, pp. 301–315.
- [3] J. Xiong, L. Chen, M. Z. A. Bhuiyan, C. Cao, M. Wang, E. Luo, and X. Liu, "A secure data deletion scheme for iot devices through key derivation encryption and data analysis," *Future Generation Computer Systems*, vol. 111, pp. 741–753, 2020.
- [4] S. Gnatyuk, V. Kinzeryavyy, T. Sapozhnik, I. Sopilko, N. Seilova, and A. Hrytsak, "Modern method and software tool for guaranteed data deletion in advanced big data systems," in *Advances in Artificial Systems for Medicine and Education II 2*. Springer, 2020, pp. 581–590.
- [5] B. Lee, K. Son, D. Won, and S. Kim, "Secure data deletion for usb flash memory," *J. Inf. Sci. Eng.*, vol. 27, no. 3, pp. 933–952, 2011.
- [6] L. Yang, T. Wei, F. Zhang, and J. Ma, "Sadus: Secure data deletion in user space for mobile devices," *computers & security*, vol. 77, pp. 612–626, 2018.
- [7] Z. Ratliff, W. Goh, A. Wieland, J. Mickens, and R. Williams, "Holepunch: Fast, secure file deletion with crash consistency," *Cryptology ePrint Archive*, 2023.
- [8] D. Zheng, L. Xue, C. Yu, Y. Li, and Y. Yu, "Toward assured data deletion in cloud storage," *IEEE Network*, vol. 34, no. 3, pp. 101–107, 2020.
- [9] C. Yang, X. Chen, and Y. Xiang, "Blockchain-based publicly verifiable data deletion scheme for cloud storage," *Journal of Network and Computer Applications*, vol. 103, pp. 185–193, 2018.
- [10] F. Hao, D. Clarke, and A. F. Zorzo, "Deleting secret data with public verifiability," *IEEE Transactions on Dependable and Secure Computing*, vol. 13, no. 6, pp. 617–629, 2015.
- [11] J. Ma, M. Wang, J. Xiong, and Y. Hu, "Cp-abe-based secure and verifiable data deletion in cloud," *Security and Communication Networks*, vol. 2021, pp. 1–14, 2021.
- [12] B. Hall and M. Govindarasu, "An assured deletion technique for cloud-based iot," in *2018 27th International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 2018, pp. 1–9.
- [13] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 141–159.
- [14] T. Eisenhofer, D. Riepel, V. Chandrasekaran, E. Ghosh, O. Ohri-menko, and N. Papernot, "Verifiable and provably secure machine unlearning," *arXiv preprint arXiv:2210.09126*, 2022.
- [15] H. Luo, B. Kuang, S. Guo, L. Zhang, B. Niu, and F. Li, "Cross-domain multi-copy of flow discovery mechanism based on dual certificate storage," *Journal on Communications*, vol. 45, no. 2, p. 40, 2024.
- [16] S. P. Huber, S. Zoupanos, M. Uhrin, L. Talirz, L. Kahle, R. Häuselmann, D. Gresch, T. Müller, A. V. Yakutovich, C. W. Andersen *et al.*, "Aiida 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance," *Scientific data*, vol. 7, no. 1, p. 300, 2020.
- [17] H. Cho, L. Zhang, and X. Jiang, "Secure cryptographic technology framework for data element circulation transactions," in *2024 IEEE 11th International Conference on Cyber Security and Cloud Computing (CSCloud)*. IEEE Computer Society, 2024, pp. 187–193.
- [18] S. Knight, H. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," *Selected Areas in Communications, IEEE Journal on*, vol. 29, no. 9, pp. 1765–1775, october 2011.
- [19] Y. Li, S. Guo, Q. Song, Y. Wang, X. Wei, and J. Ma, "Mapping prediction with recurrent neural networks for future lisp enabled networks," *Journal of Information and Intelligence*, vol. 1, no. 2, pp. 134–147, 2023.
- [20] International Data Spaces Association. (2024) International data spaces association github repository. Accessed: 2024-06-13. [Online]. Available: <https://github.com/International-Data-Spaces-Association>